EECS 348 Group 10

Arithmetic Expression Evaluator
Software Development Plan
Version <1.0>

Revision History

Date	Version	Description	Author
02/16/2026	1.0	Initial commits / group discussion	Tate Meyer
02/18/2026	1.0	Project overview is complete	Jaycob Campos
02/21/2026	1.0	Introduction (Section 1) complete; 1.4 requires revision.	Ayushma Koirala
02/22/2026	1.0	Final Overview	Maddi Clark
02/22/2026	1.0	Finishing touches to section 4, formatting	Tate Meyer

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 5

1.5 Overview 5

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 5

2.3 Project Deliverables 6

2.4 Evolution of the Software Development Plan 6

3. Project Organization 6

3.1 Organizational Structure 6

3.2 External Interfaces 6

3.3 Roles and Responsibilities 7

4. Management Process 7

4.1 Project Estimates 7

4.2 Project Plan 7

4.3 Project Monitoring and Control 8

4.4 Requirements Management 8

4.5 Quality Control 8

4.6 Reporting and Measurement 8

4.7 Risk Management 9

4.8 Configuration Management 9

5. Annexes 9

Software Development Plan

1. Introduction

This document presents the Software Development Plan for the Arithmetic Expression Evaluator project. It outlines the overall approach, structure, and management strategy that will guide the successful development and delivery of the project.

1.1 Purpose

The purpose of this software development plan is to lay out all the requirements and deadlines of the project. This includes the scope, deliverables, organizational structure, team members and their roles, project plan, and how the project will be managed. This will be used by the project manager to schedule and control the project. This will also be used by the team members for reference to what needs to be done.

1.2 Scope

This Software Development Plan applies to the development of the Arithmetic Expression Evaluator in C++. The system will parse and evaluate arithmetic expressions containing numeric constants and the following operators:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulo (%)
- Exponentiation (**)
- Unary + and -

The program will correctly implement operator precedence, associativity rules, and parenthesis handling. It will include proper error handling for invalid expressions and runtime errors such as division by zero.

This document covers the full development lifecycle of the project including planning, requirements analysis, design, implementation, testing, documentation, and delivery

1.3 Definitions, Acronyms, and Abbreviations

- **Parser** – A component that analyzes an input expression and determines its structure according to defined rules.
- **Tokenizer** – A module that breaks the input string into meaningful symbols (tokens) such as numbers and operators.
- **Operator Precedence** – The rules that determine the order in which operations are performed.
- **Associativity** – The rule that determines the order of evaluation when operators have the same precedence (left-to-right or right-to-left).
- **PEMDAS** – Parentheses, Exponents, Multiplication, Division, Addition, Subtraction.
- **CLI** – Command Line Interface

1.4 References

- Software Development Plan Template, published by UPEDU
- EECS 348 Term Project Description – Arithmetic Expression Evaluator in C++ , published by KU EECS Department

1.5 Overview

This Software development plan is organized into 5 sections.

- Section 1. Introduction
  - This gives an idea of what the purpose and scope of the project is. This also includes various definitions and references that will be used throughout the document.
- Section 2. Project Overview
  - This section gives an idea of the objectives of the project, any assumptions or restrictions, deliverables and their deadlines, and how this document will evolve through the project
- Section 3. Project Organization
  - This section shows how the project will be organized, any external interfaces to be used, and the roles, responsibilities, and contact info of the team members.
- Section 4. Management Process
  - This section will describe how the project will be managed. This includes estimates, a general plan, how it will be monitored, requirements, quality assurance, reporting, risk management, and configuration.
- Section 5. Annexes
  - This section will include any additional information added on.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

The purpose of this project is to design and develop an arithmetic expression evaluator as a component of a larger compiler product for language L. The parser will be built in C++ and will be able to parse and compute arithmetic expressions following PEMDAS. The scope of the project covers design, implementation, testing, and documentation for the arithmetic parser. It includes the operators +, -, *, /, %, and **, unary operators, parenthetical grouping, numbers, and error handling. The objective is to develop a C++ program that correctly evaluates all valid arithmetic expressions and returns clear error messages for invalid inputs. All required project artifacts should align with one another throughout the development process. Deliverables include a project management plan, requirements documents, design documents, test plan, a well-documented C++ program, and a user manual with usage examples.

2.2 Assumptions and Constraints

Assumption is that all members have a working knowledge of C++ and object-oriented programming basics. The inputs for the parser will only contain operators and operands specified in the project requirements. Team members will have constant access to the GitHub repository and development environments that support C++. Constraints are the time we must complete this arithmetic parser. The equipment is restricted to personal machines or KU computers. The arithmetic parser has six operators and doesn’t need to support variables or functions. The project must be coded in C++.

2.3 Project Deliverables

Project artifacts that will be created during the project are project management plan, requirements document, design document, test plan, C++ source code, user manual/ README. All deliverables will be submitted by the GitHub repository by the end of the specified project phase. Definitions for the deliverables can be found in section 4.2 with deadlines specifically being defined in section 4.2.4.

2.4 Evolution of the Software Development Plan

Version	Description	Date
1.0	Initial Commits	Feb. 22
1.1	Pre Design and Architecture	Mar. 15
1.2	Pre Implementation	Apr. 5
1.3	Pre Testing	May 7

3. Project Organization

3.1 Organizational Structure

The project team consists of six members working collaboratively under the UPEDU framework. While each member has an assigned primary role, the overall structure is cooperative, allowing flexibility and shared responsibility throughout development.

- The Project Manager oversees coordination, scheduling, and progress tracking.
- The Software Architect provides guidance on system design decisions.
- Test Engineers focus on verification and quality assurance.
- Documentation Managers maintain and organize project documentation.

The course instructor and teaching assistants serve as external review authorities, providing feedback and evaluating project deliverables.

3.2 External Interfaces

The primary external stakeholders for this project are the course instructor and the teaching assistants. They provide guidance, clarify requirements, and evaluate deliverables throughout the semester. Most communication with them will take place during class/lab sessions or through email if additional clarification is needed.

GitHub will serve as the team's primary collaboration and version control platform. It will function as the central repository for source code, documentation, issue tracking, and submission of deliverables. Using GitHub consistently ensures that all team members are working with the most up-to-date project materials. Additional team communication will occur through Microsoft Teams and scheduled group meetings.

As this is an academic project, final acceptance of the product will be determined by the instructor based on the grading rubric and the stated project requirements.

3.3 Roles and Responsibilities

Person	Unified Process for Education Role
Tate Meyer (tatemeyer04@ku.edu)	Project Manager / Programmer
Avery Richardson (avery.r.richardson@ku.edu)	Test Engineer / Programmer
Nikki Sotoodi (n052s368@ku.edu)	Test Engineer
Maddi Clark (cclar@ku.edu)	Documentation Manager
Ayushma Koirala (a371k869@ku.edu)	Documentation Manger
Jaycob Campos (j010c700@ku.edu)	Software Architect/Designer

Anyone on the project can perform Any Role activities.

4. Management Process

4.1 Project Estimates

The total estimated effort for this project spans roughly 1 semester, the largest time investments are expected in implementation (~35%) and testing (~25%), followed by design (~20%), documentation (~15%), and planning (~5%). These estimates assume each member contributes 3-5 hours per week on average.

4.2 Project Plan

4.2.1 Phase Plan

Phase 1 – Planning, this phase covers initial team organization, role assignment, and the creation of this Software Development Plan. The team establishes communication norms, sets up the GitHub repository, and aligns on the scope of the project. Deliverable: complete SDP.
Phase 2 – Requirements and Design, this phase produces the requirements document, initial user manual draft, and the design and architecture document. The software architect leads the system design, defining how the tokenizer, parser, and evaluator components will interact. The team reviews and approves all requirements before implementation begins. Deliverables: requirements document, system design document, initial user manual.
Phase 3 – Implementation, development of the C++ program beings once the core design is stable. This phase overlaps slightly with Phase 2 to allow early prototyping. The first iteration focuses on core functionality: tokenizing input, parsing expressions, and correctly evaluating all operators with proper precedence and associativity. Subsequent iterations address bug fixes and any additional features. Deliverable: C++ source code (beta and final releases).
Phase 4 – Testing and Finalization, test engineers develop and execute the test plan, covering valid expressions, edge cases, and error handling scenarios. Defects are logged, resolved, and re-tested per the QC process in section 4.5. Documentation is finalized. Deliverables: test plan, test results, final user manual, final source code.

4.2.2 Iteration Objectives

This project will have at least 2 iterations. The first iteration will consist of the first draft of basic features (See section 1.2). Any iterations after will be bug fixes and additional features (UI, saved history, etc).

4.2.3 Releases

- Release 1, beta:
  - Contains all operators described in section 1.2, iterates in the correct order, produces the correct answer to the provided expression
- Release 2, final release:
  - Any required bug fixes, additional features

4.2.4 Project Schedule

March 15 – Requirements complete and initial iteration of the user manual

April 5 – Design and Architecture done

May 7 – Final implementation, final user manual, final test cases completed

4.2.5 Project Resourcing

Personnel
The project consists of six members, each contributing an estimated 3-5 hours per week over the course of the project. Roles are defined in section 3.3, though all members are expected to contribute to implementation and testing as needed. The project manager is responsible for ensuring workload is distributed evenly and that no single member is over or under burdened.
Development Environment
All development will be done in C++ using standard compilers available on personal computers or KU lab computers. No paid software licenses are required. Members are responsible for maintaining a working local development environment compatible with the project's codebase.
Version Control and Collaboration
GitHub will serve as the central repository for all project artifacts including source code, documentation, and deliverables. Microsoft Teams will be used for communication and meeting coordination.

4.3 Project Monitoring and Control

All monitoring and control plans are described in the sections below (section 4.4 – 4.8).

4.4 Requirements Management

Requirements will be documented and reviewed by all team members before being baselined. Any proposed changes to requirements after the March 15 deadline must be raised in a team meeting, approved by the team, and reflected in an updated version of the requirements document. The impact of any change on the schedule and design must be assessed before approval.

4.5 Quality Control

Defects will be recorded and documented by testers and documentation managers after each iteration. Before each iteration, these defects will be reviewed to ensure that they have been fixed. If not, they will be recorded and fixed in a future iteration to be reviewed again later. Before each release these defects will be tested again to ensure they've been fixed.

4.6 Reporting and Measurement

Progress will be measured against the project schedule in 4.2.4. At each milestone the team will assess whether all required deliverables are completed and whether known defects from section 4.5 have been resolved. A brief status summary will be recorded in the GitHub repository (README update) after each milestone to maintain a record of project health.

4.7 Risk Management

Project risks will be discussed with the team during meetings. They will be recorded and monitored by the documentation managers and project manager but should be kept in mind by the whole team.

4.8 Configuration Management

GitHub will be used alongside other tools to provide a database of defects to be fixed and a controlled versioned repository of project artifacts.

All source code, test files, and other files will also be included in this GitHub repository. This includes all planning documents and other documentation of team activities. All deliverables will also be saved here.

5. Annexes

The project will follow the UPEDU process.